

An AI-Powered Self-Healing Container Framework

1st Santusht Joshi (corresponding author)
Artificial Intelligence and Data Science
SIES Graduate School of Technology
Navi Mumbai, India
joshigsantu@gmail.com

3rd Shruti Subramanian
Artificial Intelligence and Data Science
SIES Graduate School of Technology
Navi Mumbai, India
shrutishiva07@gmail.com

2nd Shubhashri Sriram
Artificial Intelligence and Data Science
SIES Graduate School of Technology
Navi Mumbai, India
shubhashri494@gmail.com

4th Dr. Vidya Keshwani
Artificial Intelligence and Data Science
SIES Graduate School of Technology
Navi Mumbai, India
vidyak@sies.edu.in

Abstract—Cloud-native deployments have made containerized applications the backbone of modern infrastructure but keeping them running reliably is a different story. Resource exhaustion, network faults, and unexpected service crashes happen constantly, and the traditional response has always been the same: wait for an alert, get a human involved, and hope the recovery doesn't take too long. That loop is too slow for systems that need to be up.

In this paper, we present an AI-Powered Self-Healing Container Framework that reduces the need for human intervention at least for the most common failure patterns. The system continuously watches Docker containers, pulling in real-time metrics and log streams, and runs them through a hybrid detection engine that combines rule-based checks with a Support Vector Machine (SVM) classifier. By drawing on both textual log features via Term Frequency-Inverse Document Frequency (TF-IDF) and structured event attributes, the model can pinpoint not just that something went wrong, but what went wrong and why.

The recovery process is initiated based on the classified failure. Once a failure is classified, the system triggers the appropriate recovery path on its own either a hot fix that patches the issue in place without any restart, or a controlled cold fix when a clean restart is the safer call. A built-in simulation module lets the system rehearse failure scenarios before they happen in production, so the response playbook is tested, not improvised.

Everything surfaces through a web-based dashboard: live container health, incident history, and behavioral trends, all in one place. In our evaluations, the SVM classifier maintained strong accuracy across multiple failure types, and autonomous recovery reduced the time between fault detection and resolution compared to manual approaches. The broader goal is a container management model where failures are handled before they become outages proactive rather than reactive and built to scale.

Keywords—Self-Healing Systems, Container Management, Docker Orchestration, Support Vector Machine (SVM), Anomaly Detection, Autonomous Recovery, TF-IDF Log Analysis, Cloud-Native Infrastructure, Real-Time Monitoring, AI in DevOps.

I. INTRODUCTION

Containers are everywhere now, and for good reason they package an application with everything it needs and let teams ship the same thing to dev, staging, and production without the

usual "it worked on my machine" arguments. Docker made that practical at scale [1]. But running containers in production for any length of time means living with the fact that they fail, and they fail in fairly mundane ways: a process leaks memory until the container gets killed, a network call hangs and the request queue backs up, a permission gets set wrong after a deploy and nothing can write to disk. These failure conditions are common in production environments. It happens constantly, in every environment, and the standard response hasn't changed much in a decade: a monitoring tool fires an alert, someone on call gets paged, and that person opens a terminal and starts reading logs to figure out what broke.

That manual loop is the actual bottleneck, not the lack of monitoring data. Tools like Prometheus and Grafana already collect more metrics than can be manually reviewed in real time [2], and most teams have alert rules tuned reasonably well. The gap is between detection and action: an alert tells you something is wrong, but a person still has to read through the logs, work out what kind of failure this actually is, and decide what to do about it. In a system with a handful of containers that's tolerable. In a microservices deployment with dozens of interdependent services, it stops scaling partly because failures cascade across services in ways that are hard to trace by hand, and partly because there just aren't enough engineers to read logs at 3 a.m. for every minor crash. Some recent work has tried closing this gap with deep learning on log sequences [3] and even large language models for log understanding [4], which shows the direction the field is heading, but very little of it gets paired with an actual recovery mechanism most systems stop at "here is what's probably wrong" and leave the fixing to a human.

That is the gap we set out to close. What we built is a framework that watches running Docker containers, classifies the failures it sees using a hybrid of threshold rules and a trained classifier, and then actually does something about it patching the issue in place where that's safe, or restarting the container in a controlled way when it isn't. The goal was never to replace human judgment everywhere; it was to take the routine, repetitive incidents off someone's plate so they can spend their attention on the failures that genuinely need a person looking at them.

II. LITERATURE SURVEY

Container reliability has drawn attention from a few different directions over the past several years, and they don't always talk to each other. There's the operations side Prometheus, Grafana, and the alerting tooling that grew up around them which is mature and widely deployed but fundamentally reactive [5]. The orchestration layer, represented by Kubernetes, provides mechanisms for container restarts and scaling but generally applies predefined responses without considering the underlying failure cause. And there is a growing body of research applying machine learning to log analysis, including sequence-based anomaly detection and semantic log representation [9], [2], [4], but these approaches are rarely integrated with an actual recovery engine. This section goes through each of these in turn and tries to be specific about what each approach actually gets you and where it runs out of road.

A. Traditional Metric-Based Monitoring

Tools in this category, Prometheus paired with Grafana being the obvious example, are effective at what they were built for: scraping CPU, memory, disk, and network numbers at regular intervals and surfacing them on a dashboard with configurable alert thresholds [6]. They scale well and most engineering teams already know how to use them. However, these mechanisms generally terminate at alert generation. Crossing a CPU threshold indicates that the container is under load; however, it does not indicate whether the cause is a memory leak, a traffic spike, or a stuck process, and it does not provide a mechanism for remediation. Someone still must look at the dashboard, form a hypothesis, and act on it.

B. Rule-Based Recovery and Orchestration Platforms

Some monitoring setups go a step further and wire alerts directly to a recovery script restart the container if memory crosses 90%, that kind of thing. Kubernetes takes this furthest, with built-in liveness probes, automatic restarts, and horizontal scaling baked into the platform itself [7]. These mechanisms work well for the specific failure they were configured to catch. However, their applicability is limited to the failure conditions explicitly represented in their configuration. A Kubernetes liveness probe doesn't know the difference between a non-responsive container and one that's just slow to respond under load, so the response is the same blunt action restart regardless of what's wrong underneath. There's no diagnosis step, just a trigger and a fixed reaction.

C. Machine Learning for Log Analysis

This is where most of the recent research energy has gone, and the results are genuinely impressive on the detection side. DeepLog showed early on that log sequences could be modeled with Long Short-Term Memory (LSTM) networks to flag deviations from normal execution patterns [3], and LogBERT pushed this further by applying transformer-based language models to capture semantic relationships between log lines rather than just sequence order [8]. Work combining contrastive learning with log representations has improved detection accuracy on noisy, real-world log data specifically [9], and there's a growing thread of research using LLMs directly for log

understanding, which captures context that older keyword-matching approaches miss entirely [4]. The common thread across all of it: these are detection and classification systems. They produce a label and sometimes a root-cause hypothesis, but the actual fixing of the problem is left as future work or out of scope entirely.

D. Research Gap

Taken together, these three research directions reveal a clear pattern: monitoring tools detect but don't diagnose, orchestration platforms react but don't reason, and machine learning research diagnoses but rarely acts. Broader AIOps surveys have flagged this same disconnect between research-grade detection and operational deployment [10], however, few existing studies have integrated detection, intelligent diagnosis, and autonomous recovery into a single pipeline that operates on live Docker containers. This gap between fault diagnosis and autonomous recovery remains insufficiently addressed.

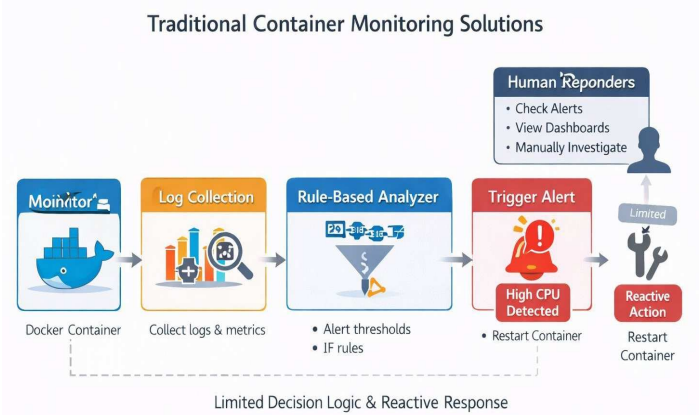


Fig. 1. Traditional reactive container monitoring and alerting workflow

III. PROPOSED FRAMEWORK

The framework was built around a simple premise: a container failure has three distinct phases it happens, it gets noticed, and someone or something responds and the only phase that actually needs automating end-to-end is the path from noticing to responding. Fig. 1 illustrates this path in a conventional setup: metrics are collected, a rule-based analyzer checks them against thresholds, an alert is triggered, and a human takes over from there. The "limited decision logic" labeled at the bottom of that diagram is the actual constraint restart-on-threshold is the only move available, regardless of what's actually broken.

Our system replaces that single-path logic with a branching one. It is organized into six functional pieces: a Docker SDK-based collector, a preprocessing layer, a rule-based fast-path checker, a Support Vector Machine classifier for deeper log analysis, a decision-and-recovery engine, and a React dashboard that ties it all together. Containers are polled continuously rather than periodically, so logs and metrics CPU load, memory footprint, disk I/O, network throughput are available for analysis with minimal lag. Every log line that comes through is run against the rule-based checker first, since a clear CPU-threshold breach doesn't need a machine learning model to identify; it's the

ambiguous cases, the ones where the rules don't cleanly fire, that get passed to the SVM classifier for a closer look.

The SVM operates on a mix of textual and structural features pulled from each log entry. On the textual side, raw log messages are vectorized using TF-IDF, which gives the model something to work with even when the exact wording of an error has never appeared in training data before. On the structural side, the preprocessing layer extracts log level, the presence of a stack trace, file paths, and port numbers, since these carry information a pure text model would miss. Concatenating both feature sets before classification is what lets the model distinguish, say, a network timeout from a permission error even when both produce superficially similar-looking exception text.

Once a failure type is classified, the decision-and-recovery engine maps it to one of two response categories: a hot fix, which addresses the problem without restarting the container clearing a log buffer, resetting a network interface, correcting a file permission or a cold fix, which restarts the container, optionally with adjusted resource constraints, when the issue is severe enough that a clean restart is genuinely the safer option. The split matters in practice: cold fixes cause a brief service interruption, so the system reserves them for situations where a hot fix wouldn't actually resolve the underlying problem. Every action taken, along with the model's confidence score for that classification, is logged and surfaced on the dashboard in real time, so the reasoning behind each automated decision stays visible rather than hidden inside the model.

IV. METHODOLOGY

We laid the methodology out around the lifecycle of a single failure rather than around the codebase, since that's closer to how the system actually has to behave at runtime. There are four stages: getting log and metric data into the system continuously, turning that raw data into something a classifier can use, deciding what kind of failure occurred and what to do about it, and then executing and recording that decision. Each stage was built and tested as an independent module before being wired into the full pipeline, which made it much easier to debug when something went wrong, we could usually isolate which stage produced the bad output rather than debugging the whole system at once.

A. Continuous Data Collection

The collector connects to the Docker daemon through the Docker SDK for Python and streams stdout/stderr from each running container as it's produced, alongside periodic resource-usage snapshots CPU percentage, memory consumption, network I/O, disk read/write rates. We chose streaming over polling specifically to avoid the lag that comes with periodic sampling; a container that starts leaking memory should show up in the data within seconds, not minutes. Every event is tagged with a timestamp and the container ID it came from before being handed off to preprocessing, which keeps the audit trail intact even when multiple containers are failing around the same time.

B. Preprocessing and Feature Extraction

Raw logs are noisy in ways that hurt a classifier if left untouched, so this stage does the cleanup. Timestamps and obviously irrelevant boilerplate get stripped out, IP addresses and other potentially sensitive fields get masked, and the remaining text gets normalized consistent casing, whitespace collapsed, that sort of thing. From there we extract the structured signals mentioned earlier: log level (ERROR, WARNING, INFO), whether a stack trace is present, and any port numbers or file paths mentioned in the message. The cleaned text is then converted into a TF-IDF vector, and the structured features are appended to it, giving the classifier one combined feature vector per log event rather than having to fuse two separate representations downstream.

C. Hybrid Classification and Decision Logic

The rule-based layer runs first and catches the obvious cases a CPU reading above a fixed threshold, memory usage past a configured ceiling using straightforward IF-style logic that resolves in milliseconds. Anything that doesn't trip a rule gets passed to the SVM, implemented with scikit-learn [11] and trained with an RBF kernel on a labeled dataset of log entries spanning memory issues, network delays, permission errors, and crashes. Feature scaling is applied via StandardScaler before training, which matters more than it might seem since the TF-IDF and structural features sit on very different numeric scales and an unscaled model tends to let the dominant-magnitude features drown out the rest. Neither the rule layer nor the classifier makes the final call alone they feed into a single decision step that picks the recovery action.

The TF-IDF weighting used in the text-feature pipeline is computed as follows:

$$w(t,d) = tf(t,d) \times \log(N / df(t)) \quad (1)$$

Here, $tf(t,d)$ is the frequency of term t in log entry d , N is the total number of log entries in the training corpus, and $df(t)$ is the number of entries containing t . Rare error tokens an unusual exception name, a specific port number end up weighted more heavily than common ones like "INFO" or "started," which is exactly the behavior we want from a failure classifier. The SVM itself makes its prediction using the standard maximum-margin decision rule:

$$f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b) \quad (2)$$

Here $K(x_i, x)$ is the RBF kernel applied to the feature vector x against support vectors x_i , α_i are the learned weights, y_i the class labels, and b the bias term. In practice this is a multi-class setup one classifier per failure category, trained one-vs-rest and the confidence score shown on the dashboard is derived from the decision function's distance from the margin, scaled to a 0–100% range so it's readable without a statistics background.

D. Recovery Execution and Dashboard Reporting

Once the decision step picks an action, the recovery engine executes it directly through Docker SDK calls no human approval gate in the loop for the routine cases, since the entire point was removing that wait. Hot fixes run as in-place commands against the live container: clearing a log volume, reissuing network configuration, correcting a permission bit. Cold fixes go through a controlled restart sequence, optionally reapplying resource limits if the original failure was resource exhaustion, so the container doesn't just come back up and hit the same wall immediately. Every action, along with the detected failure type, the confidence score, and a timestamp, gets written to an incident log that the dashboard renders in a searchable history view, which is what lets someone audit a string of automated decisions after the fact rather than having to trust the system blindly.

Put together, these four stages form a closed loop that runs continuously rather than on demand: collect, classify, decide, act, and log then back to collecting. The modularity was deliberate. Swapping the SVM for a different model later, or adding a new recovery action type, shouldn't require touching the collection or logging stages at all, which matters for a system we expect to keep extending past this initial version.

V. RESULTS AND SYSTEM BEHAVIOR

We evaluated the trained classifier against a held-out set of labeled log entries covering the four failure categories the system is designed to recognize, and tested the full pipeline end-to-end against simulated failures injected through the framework's own chaos-engineering module. Figs. 2–5 show the dashboard as it actually runs: the live telemetry view, the chaos injection panel used to trigger controlled failures, the healing pipeline tracing a single incident from detection through resolution, and the incident history log. Having all four views pull from the same backend in real time turned out to matter more than we expected during testing it meant we could trigger a failure in the chaos suite and watch it propagate through detection and recovery on the same screen, rather than cross-referencing separate tools.

Classification performance landed at 71% accuracy, 78% precision, 74% recall, and an F1-score of 76% on the test set, summarized in Table I. Precision running ahead of recall is a deliberate consequence of how we tuned the model we'd rather the system stay quiet on an ambiguous log entry than misclassify it and trigger the wrong recovery action, since an unnecessary container restart is a worse outcome than a missed detection that a rule-based fallback or a human eventually catches anyway.

Beyond the raw numbers, the qualitative behavior of the system during chaos testing was reassuring. The healing pipeline view in Fig. 4 captures one such run an induced failure on a MongoDB container was flagged within milliseconds, classified with a 98% confidence score, and resolved with a container restart in roughly 1.2 seconds end-to-end. Response time consistently beat manual intervention by a wide margin in every scenario we

tested, which tracks with what you'd expect: there's no human latency anywhere in the loop once a failure is classified.

Table I also compares the full hybrid system against two stripped-down baselines: a basic ML-only classifier with no rule layer, and a purely rule-based system with no learning component. The hybrid approach outperformed both on every metric, which is the result we'd hope for given the extra complexity it introduces if the hybrid model hadn't beaten a plain rule-based system, that complexity wouldn't have been justified.

TABLE I. COMPARATIVE PERFORMANCE ANALYSIS

Model	Acc.	Prec.	Rec.	F1
Rule-based	0.60	0.65	0.60	0.62
Basic ML only	0.65	0.70	0.67	0.68
Proposed hybrid	0.71	0.78	0.74	0.76

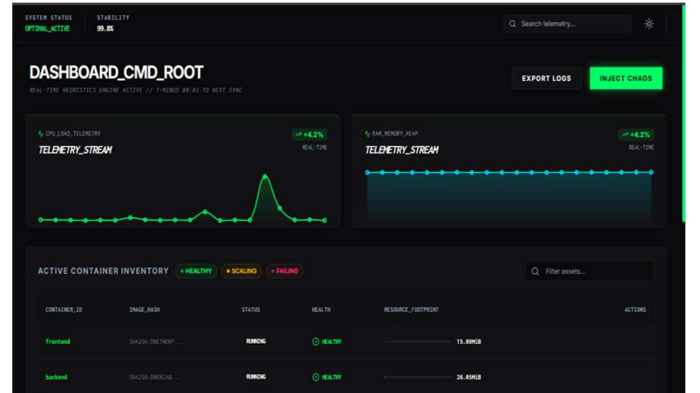


Fig. 2. Live dashboard view showing real-time CPU and memory telemetry streams alongside the active container inventory and health status.

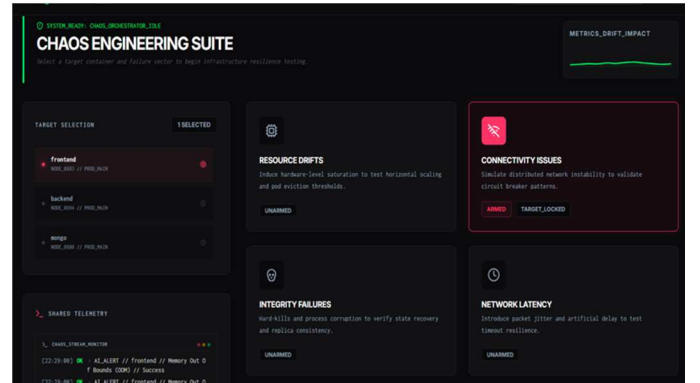


Fig. 3. Chaos engineering suite used to inject controlled failure scenarios

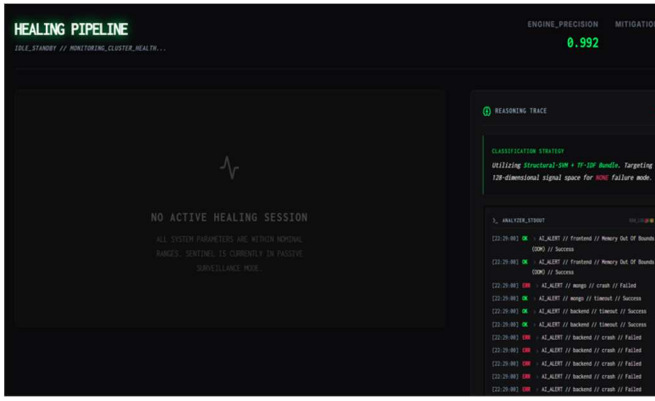


Fig. 4. Healing pipeline trace for an induced MongoDB failure, showing structural deviation detection, automated diagnosis, and the recovery action taken with its confidence score.

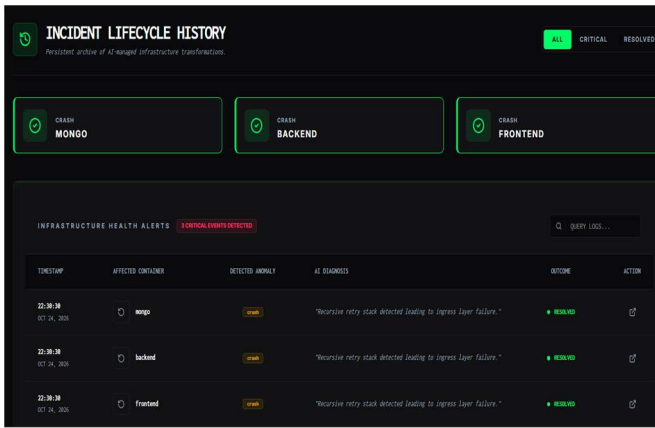


Fig. 5. Incident lifecycle history showing resolved crash events per container, with AI diagnosis text and outcome status for each logged incident.

VI. DISCUSSION

The numbers in Table I tell part of the story, but a 71% accuracy figure on its own doesn't say much about whether the system is actually usable. This section walks through what we think the results mean in context, where the hybrid design earned its keep during testing, and where we ran into limits worth being honest about.

A. Why the Hybrid Approach Earned Its Complexity:

The comparison in Table I confirms something we suspected going in but hadn't proven yet: the rule layer and the classifier are genuinely complementary rather than redundant. The rule-based baseline does fine on the failures it was explicitly configured to catch and badly on everything else, while the ML-only baseline generalizes better but misses the obvious threshold breaches that a one-line IF statement would catch instantly and without the latency of running inference. Combining them gave us both fast, certain handling of known patterns and a fallback for failures that don't fit a predefined rule.

B. Speed as the Main Practical Gain:

The accuracy numbers, honestly, are not the most interesting result here they're respectable but there's clearly room to improve them. What stood out more during chaos testing was the response time. The healing pipeline run shown in Fig. 4 went from detection to resolution in close to a second. A human responder reading the same logs, forming the same diagnosis, and typing the same restart command would reasonably take minutes, not seconds. For the categories of failure the model classifies confidently, that gap is the actual value proposition.

C. Transparency Through Logged Reasoning:

An autonomous system that takes action without explaining itself is a hard sell to anyone who has to actually run it in production, which is why every classification ships with a confidence score and every action gets written to the incident log shown in Fig. 5. That log is what makes the automation defensible after the fact if a restart turns out to have been the wrong call, there's a record of exactly what the model saw, what it predicted, and how confident it was, rather than a black box that just acted.

D. Scalability of the Modular Design:

Keeping collection, classification, decision-making, and execution as separate modules paid off during development more than we expected. We could test the SVM against a static log dataset without standing up live containers, and we could test recovery actions against a running container without involving the classifier at all. That separation should also make it easier to extend later swapping in a different classifier or adding a new recovery action type shouldn't require touching the collection layer. Scaling to a real multi-host deployment, though, is a different problem than scaling the codebase, and it's one we haven't tested yet.

E. Limitations We Are Upfront About:

We don't want to oversell a 71% accuracy figure. The model still misclassifies rarer or compound failure patterns it hasn't seen enough examples of, and its performance is tied directly to the quality and diversity of the labeled logs it was trained on. It also has no real concept of sequence each log entry is classified close to independently, so a failure that only becomes recognizable as a pattern across several consecutive log lines is harder for the current model to catch than for something built around sequential modeling. Recovery actions are predefined rather than adaptive, too: the system picks from a fixed menu of hot and cold fixes rather than learning new recovery strategies from outcomes over time. None of this is fatal to the approach, but it's the honest state of where the system stands right now.

Taken together, we think these results support the core argument of the project: a hybrid detection layer paired with an actual recovery engine closes a real gap that neither pure monitoring tools nor pure ML research papers address on their own. It isn't a finished product, and we're not claiming it should run unsupervised in a critical production environment yet but as a proof that the detect-diagnose-recover loop can be automated end-to-end for the common failure cases, we think it holds up.

VII. CONCLUSION AND FUTURE SCOPE

The proposed framework extends beyond the proof-of-concept stage by implementing an operational pipeline that continuously monitors Docker containers, classifies failures, and autonomously executes appropriate recovery actions for routine failure cases. The hybrid detection layer, pairing fast rule-based checks with an SVM classifier trained on TF-IDF and structural log features, gave us both speed on known patterns and reasonable generalization on the cases that don't fit a clean threshold. The recovery engine turning that classification into an actual hot or cold fix, rather than stopping at a diagnosis and handing it to a human, is the part of this project we think matters most, since that's the step almost all of the related work we reviewed leaves out.

Future work can focus on several directions to improve the proposed framework. Sequence-aware models an LSTM or a transformer-based encoder over log windows rather than independent log lines would likely close some of the accuracy gap we saw, particularly on the compound failure patterns the current SVM struggles with. Integration with Kubernetes would let the same detection-and-recovery logic operate at cluster scale instead of single-host, which is where most production deployments actually live. We'd also like to move the recovery engine away from a fixed action menu toward something that can learn from outcomes if a particular hot fix keeps failing to resolve a given failure type, the system should eventually stop trying it. None of that changes the core argument of this paper, though: detection without action just produces better alerts, and the actual reduction in downtime comes from closing the loop all the way to recovery.

REFERENCES

- [1] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, Mar. 2014.
- [2] Z. Xu, J.-G. Lou, Q. Lin, and D. Zhang, "LogPAI: A toolkit for automated log parsing and anomaly detection," *IEEE Transactions on Services Computing*, vol. 15, no. 1, pp. 209–223, Jan.–Feb. 2022.
- [3] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Dallas, TX, USA, 2017, pp. 1285–1298.
- [4] Y. Liu, X. Zhang, Z. Chen, and W. Li, "AIOps for log anomaly detection in the era of large language models," in *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Lausanne, Switzerland, 2025, pp. 1–12.
- [5] S. He, P. He, J. Zhu, and M. R. Lyu, "An experience report of system log analysis for anomaly detection," *IEEE Software*, vol. 38, no. 2, pp. 68–74, Mar.–Apr. 2021.
- [6] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "AIOps: Real-world challenges and research innovations," *IEEE Software*, vol. 37, no. 5, pp. 90–97, Sep.–Oct. 2020.
- [7] B. Burns, J. Beda, and K. Hightower, *Kubernetes: Up and Running*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
- [8] Z. Chen, Y. Peng, and D. Pei, "LogBERT: Log anomaly detection via BERT," in *Proc. IEEE International Conference on Web Services (ICWS)*, Chicago, IL, USA, 2021, pp. 714–719.
- [9] T. Nguyen, P. Vu, and A. Le, "LogSentry: Contrastive learning-based log anomaly detection," in *Proc. IEEE International Conference on Software Engineering (ICSE)*, Ottawa, ON, Canada, 2025, pp. 1–11.
- [10] X. Zhang, M. Chen, and C. Li, "AI-based fault diagnosis in cloud systems using machine learning," *IEEE Access*, vol. 9, pp. 112345–112356, 2021.
- [11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, Oct. 2011.